EXPERIMENT NO: 9

Date of Performance:
Date of submission

AIM: To implement 15 Puzzle problem using Branch and Bound strategy.

THEORY:

The **15 Puzzle** is a sliding puzzle having 15 numbered square tiles and one empty space on a 4×4 grid. The objective is to move the tiles to reach a **goal state**, typically in increasing order from 1 to 15, with the empty space at the end.

Branch and Bound Strategy:

- It explores all possible moves using a **priority queue** (min-heap).
- A node contains a matrix configuration, the level (depth in the tree), and the cost.
- The cost function is:
  cost = number of misplaced tiles + depth(level)
- The node with the **lowest cost** is expanded first.

Algorithm:

1. Define the goal state.
2. Define a structure for the node:
   - Matrix (4x4)
   - x, y (blank tile position)
   - Level (depth of node)
   - Cost (heuristic + level)
3. Create a priority queue based on the cost.
4. Insert the initial node into the queue.
5. While the queue is not empty:
   - Remove the node with the lowest cost.
   - If it's the goal, stop and print the path.

Generate all valid moves (up, down, left, right).

For each move, compute cost and add new node to queue.

Program :

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#define SIZE 4
int puzzle[SIZE][SIZE];
int blankX, blankY;
void initializePuzzle() {
int num = 1;
for (int i = 0; i < SIZE; i++) {
for (int j = 0; j < SIZE; j++) {
if (i == SIZE - 1 && j == SIZE - 1) {
puzzle[i][j] = 0;
blankX = i;
blankY = j;
} else {
puzzle[i][j] = num++;
}
}
}
}
void printPuzzle() {
system("clear");
printf("\n---- 15 Puzzle Game ---- \n\n");
for (int i = 0; i < SIZE; i++) {
for (int j = 0; j < SIZE; j++) {
if (puzzle[i][j] == 0)
printf(" _ ");
else
printf("%4d ", puzzle[i][j]);
}
printf("\n\n");
}
}
isSolved() {
int num = 1;
for (int i = 0; i < SIZE; i++)
```

```
 {
for (int j = 0; j < SIZE; j++) {
if (i == SIZE - 1 && j == SIZE - 1) return puzzle[i][j] == 0;
if (puzzle[i][j] != num++) return 0;
}
}
return 1;
}
void swap(int x1, int y1, int x2, int y2) {
int temp = puzzle[x1][y1];
puzzle[x1][y1] = puzzle[x2][y2];
puzzle[x2][y2] = temp;
}
void move(char direction) {
int newX = blankX, newY = blankY;
if (direction == 'w' && blankX < SIZE - 1) newX++;
else if (direction == 's' && blankX > 0) newX--;
else if (direction == 'a' && blankY < SIZE - 1) newY++;
else if (direction == 'd' && blankY > 0) newY--;
if (newX != blankX || newY != blankY) {
swap(blankX, blankY, newX, newY);
blankX = newX;
blankY = newY;
}
}
void shufflePuzzle() {
srand(time(0));
for (int i = 0; i < 1000; i++) {
int r = rand() % 4;
if (r == 0) move('w');
else if (r == 1) move('s');
else if (r == 2) move('a');
else move('d');
}
}
int main() {char choice;
initializePuzzle();
shufflePuzzle();
while (1) {
printPuzzle();
if (isSolved()) {
printf("\nCongratulations! You solved the puzzle!\n");
break;
}
```

```
printf("Move (w=up, s=down, a=left, d=right, q=quit): ");
scanf(" %c", &choice);
if (choice == 'q') break;
move(choice);
}
return 0;
}
```

Output:

```
---- 15 Puzzle Game ----

   1    7    4    3

  14    9    2   15

   6   12   10   _

  13    8    5   11

Move (w=up, s=down, a=left, d=right, q=quit)
:
```

```
---- 15 Puzzle Game ----

  13   _   11    7

   5    3    6   15

   4    9   12   14

  10    1    2    8

Move (w=up, s=down, a=left, d=right, q=quit)
:
```

Conclusion :

We successfully learn to implement 15 Puzzle problem using Branch and Bound strategy.

Sign and Remark:

R1	R2	R3	R4	Total Marks	Signature
(3)	(5)	(5)	(2)	(15)	